Standard Template Library (STL) II

Universidade Estadual do Sudoeste da Bahia - UESB

2026.1

Motivação: Problema J — João João

Enunciado (simplificado):

Um professor precisa preparar uma prova com questões com **4 níveis** de dificuldades distintos (numerados de 1 a 4). Ele já possui um banco com 10 questões salvas, mas algumas podem ser da mesma dificuldade. Quantos níveis ainda estão **faltando** no banco?

Exemplo:

- **Entrada:** `1 2 1 3 2 1 3 2 1 2`
- **Saída:** `1` (o nível 4 ainda não está presente)

`std::set` — Definição

O `set` representa o **conjunto matemático**. Ou seja:

- Os termos são **únicos** — duplicatas são ignoradas automaticamente.
- **Não** é uma estrutura de acesso aleatório — não há índices como em vetores.

`std::set` — Uso e Funções

- **Declaração:** `set<tipo> s;`
- `s.size()` — quantidade de elementos no set.
- `s.insert(t)` — inserção em **O(log n)**.
  Obs.: só insere se o valor ainda não existir no set.
- `s.erase(t)` — remoção em **O(log n)**.
  Obs.: só remove se o valor existir no set.
- `s.count(t)` — retorna 1 se o elemento está no set, 0 caso contrário.
  Complexidade: **O(log n)**.

std::set — Exemplo de Uso

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    set<int> s;

    s.insert(10);  // s = {10}
    s.insert(5);   // s = {5, 10}
    s.insert(10);  // s = {5, 10}  <- duplicata ignorada

    cout << s.size() << "\n";   // 2
    cout << s.count(5) << "\n"; // 1 (existe)
    cout << s.count(7) << "\n"; // 0 (nao existe)

    s.erase(5);  // s = {10}
    s.erase(99); // nao faz nada (99 nao existe)
}
```

`std::set` — Implementação

O `set` é implementado internamente usando uma **Árvore Binária Rubro-Negra** (*Red-Black Tree*).

- Utiliza o mesmo princípio de balanceamento de uma árvore AVL, porém é **menos rígida**.
- Em uma AVL, a diferença de altura entre subárvores não pode ultrapassar 1. Na Rubro-Negra, o caminho mais longo pode ter até o **dobro** do mais curto — o que **reduz o número de rotações** necessárias por inserção.

Portanto:

- **AVL:** melhor para buscas constantes.
- **Rubro-Negra:** melhor para inserções/remoções constantes.

`std::set` — Elementos Sempre Ordenados

Uma consequência direta da implementação por árvore é que os elementos do `set` são **sempre mantidos em ordem crescente**.

```
set<int> t;
t.insert(4); t.insert(9); t.insert(2);

for (int i : t) cout << i << ' ';
// Saida: 2 4 9
```

`std::unordered_set` — Set com Tabela Hash

O `set` também pode ser implementado com uma **tabela hash** através do `unordered_set`:

```
unordered_set<tipo> us;
us.insert(t); // O(1)
us.erase(t);  // O(1)
us.count(t);  // O(1)
```

Atenção: **não utilize** o `unordered_set` se você não sabe o que está fazendo.

Motivo: colisões da função hash podem degradar a complexidade para O(n) no pior caso.

Quando usar `set`, e não `vector`?

- **Eliminar duplicatas:** o `set` descarta automaticamente valores repetidos de uma lista de elementos.
- **Inserções e remoções frequentes:** no `vector`, buscar um valor requer busca binária (**O(log n)**), mas exige que o vetor esteja ordenado (**O(n log n)**). No `set`, todas essas operações custam **O(log n)** sem pré-requisitos.

Quando usar `vector` então? Quando for necessário **acesso aleatório** aos elementos por índice.

Set ou Vector? — Exemplo 1

Situação: Você recebe uma lista de **N números inteiros** e precisa imprimir o **k-ésimo maior** deles.

- Você precisa acessar uma posição específica pelo índice?
- Há valores duplicados que devem ser mantidos?

Set ou Vector? — Exemplo 1

Situação: Você recebe uma lista de **N números inteiros** e precisa imprimir o **k-ésimo maior** deles.

- Você precisa acessar uma posição específica pelo índice?
- Há valores duplicados que devem ser mantidos?

Use `vector`! O acesso aleatório por índice é necessário, e duplicatas devem ser preservadas.

Set ou Vector? — Exemplo 2

Situação: Você recebe uma lista de palavras e quer saber **quantas palavras distintas** existem nela.

- Você precisa eliminar repetições automaticamente?
- Você precisa acessar as palavras por índice?

Set ou Vector? — Exemplo 2

Situação: Você recebe uma lista de palavras e quer saber **quantas palavras distintas** existem nela.

- Você precisa eliminar repetições automaticamente?
- Você precisa acessar as palavras por índice?

Use `set`! Basta inserir todas as palavras e consultar `s.size()`.

Set ou Vector? — Exemplo 3

Situação: Você precisa manter uma lista de IDs de usuários ativos, com **inserções e remoções** acontecendo o tempo todo, e ocasionalmente verificar se um determinado ID está ativo.

- Inserções e remoções são frequentes?
- A verificação de existência (`count`) é necessária?

Set ou Vector? — Exemplo 3

Situação: Você precisa manter uma lista de IDs de usuários ativos, com **inserções e remoções** acontecendo o tempo todo, e ocasionalmente verificar se um determinado ID está ativo.

- Inserções e remoções são frequentes?
- A verificação de existência (`count`) é necessária?

Use `set`! Inserção, remoção e busca em O(log n), sem necessidade de manter o vetor ordenado manualmente.

Resolução — João João

Enunciado (simplificado):

Um professor precisa preparar uma prova com questões com **4 níveis** de dificuldades distintos (numerados de 1 a 4). Ele já possui um banco com 10 questões salvas, mas algumas podem ser da mesma dificuldade. Quantos níveis ainda estão **faltando** no banco?

Resolução — João João

Enunciado (simplificado):

Um professor precisa preparar uma prova com questões com **4 níveis** de dificuldades distintos (numerados de 1 a 4). Ele já possui um banco com 10 questões salvas, mas algumas podem ser da mesma dificuldade. Quantos níveis ainda estão **faltando** no banco?

Solução:

Deve-se inserir os elementos em um `set`, pois assim eliminam-se os elementos repetidos e sobram apenas as dificuldades das questões que o professor já possui. Daí, basta subtrair por 4 para descobrir quantos ainda faltam.

Resolução — João João

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    set<int> s;
    for (int i = 0; i < 10; i++) {
        int n; cin >> n;
        s.insert(n);
    }
    cout << 4 - s.size() << endl;
}
```

Motivação: Contagem de Nomes

Enunciado: Em uma festa, cada pessoa registra seu nome ao entrar. Depois, você precisa responder consultas sobre quantas vezes cada nome apareceu.

Entrada:

```
7
Ana
Joao
Ana
Maria
Joao
Ana
Pedro
3
Ana
Joao
Carlos
```

Saída:

```
3
2
0
```

`std::map` — Definição

O `map` é a representação de um **dicionário**. Ou seja:

- Uma coleção de pares, onde cada um possui uma **chave** e um **valor**.
- Para consultar um valor, é necessário ter sua **chave correspondente**.
- Diferente do `set`, são as **chaves** que são únicas — e não os valores.

`std::map` — Declaração e Acesso

- **Declaração:** `map<tipo_chave, tipo_valor> m;`
  Exemplo: `map<string, int> m;`
- `m[chave]` — acessa o valor correspondente à chave **O(log n)**.
  Obs.: o acesso é semelhante ao de um array, onde no lugar do índice se utiliza a chave.

Atenção: acessar uma chave ainda não criada faz com que ela seja **criada automaticamente** com o valor padrão do tipo — aumentando o tamanho do `map`.

- `int` → 0
- `string` → `""`

std::map — Exemplo de Uso

```
map<string, int> m;

// Insercao
m["banana"] = 3; // {("banana", 3)}
m["maca"]   = 5; // {("banana", 3); ("maca", 5)}
m["banana"] = 7; // {("banana", 7); ("maca", 5)}

// Leitura
cout << m["maca"]   << "\n"; // 5
cout << m["banana"] << "\n"; // 7

cout << m["uva"] << "\n"; // 0
cout << m.size() << "\n"; // 3 (banana, maca, uva)
```

`std::map` — Outras Funções

- `m.count(chave)` — verifica se uma chave existe `(1)` ou não `(0)` no `map`. **O(log n)**.
- `m.erase(chave)` — remove a chave (e seu valor) do `map`. **O(log n)**.
- `m.size()` — retorna a quantidade de chaves instanciadas. **O(1)**.

std::map — Exemplo com Todas as Funções

```
map<string, int> m;
m["banana"] = 3;
m["maca"]   = 5;
m["banana"] = 7;

cout << m["maca"]   << "\n"; // 5
cout << m["banana"] << "\n"; // 7
cout << m["uva"]    << "\n"; // 0 (criada com valor padrao)

cout << m.count("maca")   << "\n"; // 1 (existe)
cout << m.count("melao")  << "\n"; // 0 (nao existe)

m.erase("uva");
cout << m.size() << "\n"; // 2 (banana, maca)
```

`std::map` — Implementação

O `map` também é implementado usando a **Árvore Binária Rubro-Negra**, porém seus nós são **pares (chave, valor)**.

Por causa disso, as **chaves são sempre mantidas em ordem crescente**:

```
map<int, string> m;
m[30] = "trinta";
m[10] = "dez";
m[20] = "vinte";

for (auto p : m)
    cout << p.first << ": " << p.second << "\n";
// Saida:
// 10: dez
// 20: vinte
// 30: trinta
```

Obs.: assim como o `set`, há o `unordered_map` implementado por tabela hash — porém deve ser evitado pelos mesmos motivos.

Quando usar `map`, e não `vector`?

Use o `map` quando for necessário **agrupar ou indexar termos por uma chave**.

Exemplo: organizar palavras pelo seu tamanho. Como deve ser a estrutura de dados?

- **Resposta:** `map<int, vector<string>> m;`

Onde `m[x]` retorna a lista de palavras que possuem tamanho `x`.

Qual estrutura usar? — Exemplo 1

Situação: você recebe uma lista de alunos com suas respectivas notas e precisa, dada uma consulta com o nome de um aluno, retornar rapidamente a sua nota.

Qual estrutura usar? — Exemplo 1

Situação: você recebe uma lista de alunos com suas respectivas notas e precisa, dada uma consulta com o nome de um aluno, retornar rapidamente a sua nota.

- **Resposta:** `map<string, double> notas;`

Onde `notas["Ana"]` retorna a nota da aluna Ana.

Qual estrutura usar? — Exemplo 2

Situação: você recebe eventos identificados por um código inteiro. Cada código pode ter **múltiplos horários** associados a ele. Dada uma consulta com o código, liste todos os horários desse evento.

Qual estrutura usar? — Exemplo 2

Situação: você recebe eventos identificados por um código inteiro. Cada código pode ter **múltiplos horários** associados a ele. Dada uma consulta com o código, liste todos os horários desse evento.

- **Resposta:** `map<int, vector<string>> horarios;`

Onde `horarios[42]` retorna a lista de horários do evento de código 42.

Resolução — Contagem de Nomes

Enunciado:

Em uma festa, cada pessoa registra seu nome ao entrar. Depois, você precisa responder consultas sobre quantas vezes cada nome apareceu.

Resolução — Contagem de Nomes

Enunciado:

Em uma festa, cada pessoa registra seu nome ao entrar. Depois, você precisa responder consultas sobre quantas vezes cada nome apareceu.

Solucao:

Deve-se criar um `map` onde a chave é o nome (`string`) e o valor é a frequência de aparições (`int`). Toda vez que um nome aparecer na lista, soma-se 1 à frequência de sua respectiva chave.

Resolução — Contagem de Nomes

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int N;
    cin >> N;
    map<string, int> contagem;
    for (int i = 0; i < N; i++) {
        string nome;
        cin >> nome;
        contagem[nome]++;
    }
```

Resolução — Contagem de Nomes

```
    int Q;
    cin >> Q;
    for (int i = 0; i < Q; i++) {
        string consulta;
        cin >> consulta;
        cout << contagem[consulta] << '\n';
    }
}
```

Accepted

Tempo: 0.00s | Memória: 0 KB

Nos vemos na próxima aula.